Week 13
Multi-Agent RL and AlphaZero

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

Contents

Learning objectives 4

1 Motivation and intuition 5
- 1.1 Anchoring . . . 5
- 1.2 Why this week sits where it does . . . 5

2 Mathematical formulation 7
- 2.1 Stochastic games . . . 7
- 2.2 Cooperative MARL: centralised critic, decentralised execution . . . 7
  - 2.2.1 MADDPG (Lowe et al. NeurIPS 2017) . . . 7
  - 2.2.2 MAPPO (Yu et al. NeurIPS 2022) . . . 8
  - 2.2.3 QMIX (Rashid et al. ICML 2018) . . . 8
- 2.3 Competitive MARL: self-play, fictitious play, PSRO . . . 8
  - 2.3.1 Self-play . . . 8
  - 2.3.2 Fictitious play . . . 8
  - 2.3.3 Policy-Space Response Oracles (PSRO) . . . 9

3 AlphaZero 10
- 3.1 The core idea . . . 10
- 3.2 MCTS with PUCT . . . 10
- 3.3 Dirichlet noise . . . 11
- 3.4 Training loss . . . 11
- 3.5 Symmetry augmentation . . . 11
- 3.6 Putting it together . . . 12

4 MuZero, MCTS extensions, and modern self-play 13
- 4.1 MuZero (Schrittwieser et al. Nature 2020) . . . 13
- 4.2 Stochastic MuZero and EfficientZero . . . 13
- 4.3 KataGo and the AlphaZero hyperparameter zoo . . . 13
- 4.4 Self-play in two-player zero-sum games: theory . . . 14
- 4.5 League training (AlphaStar) . . . 14

5 Algorithms 15

6 Implementation walkthrough 17
- 6.1 The network . . . 17
- 6.2 MCTS core . . . 17
- 6.3 MAPPO actor and critic . . . 18
- 6.4 The orchestration . . . 18

7 Empirical results 19

8 Safety angle 22

9 Matching application 23

10 Exercises 24

11 Further reading **25**

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Distinguish cooperative, competitive, and mixed multi-agent settings; state the centralised-training / decentralised-execution (CTDE) paradigm.
2. Derive the policy gradient for MADDPG and the centralised critic of MAPPO; explain why CTDE works even when the execution-time policy is decentralised.
3. State and discuss the QMIX monotonicity caveat: when does the factorised joint Q-function approximate the true joint Q-function?
4. Explain self-play, fictitious play, and Policy-Space Response Oracles (PSRO) as iterative best-response procedures.
5. Derive the AlphaZero MCTS-with-PUCT selection rule, the Dirichlet-noise exploration scheme, and the joint policy/value training loss.
6. Train a small AlphaZero on Tic-Tac-Toe (3x3) to near-optimal play in ~ 1 minute on CPU; observe the ELO-style winrate climb.
7. Train MAPPO on a coordination matrix game to perfect coordination, and on a SimpleSpread-style cooperative-coverage env (noisy but non-trivial).

1 Motivation and intuition

The single-agent RL we have built so far assumes the environment is stationary — the transition dynamics do not change. In multi-agent settings this is false: each other agent's policy is part of the environment, and as other agents learn, the environment shifts under you. This is the *non-stationarity problem* of MARL.

Multi-agent problems come in three flavours:

Cooperative. All agents share a reward signal. Examples: ride-hailing dispatch, factory floor coordination, multi-robot search. The algorithmic challenge is credit assignment (who contributed to the shared reward) and coordination (without explicit communication, how do agents converge on compatible policies). *This is what MAPPO solves.*

Competitive. Zero-sum: one agent's gain is another's loss. The canonical examples are board games (chess, go, poker) and adversarial robust-RL (one agent picks dynamics, the other picks policies). The algorithmic challenge is the moving-target problem: as opponent improves, the policy that beat the old opponent loses to the new one. *This is what AlphaZero solves.*

Mixed (general-sum). Some shared, some opposed interests. Two-sided matching markets (synthetic insurance brokers vs. leads, ATS recruiters vs. candidates) fall here: the platform's reward is system-level binds / hires; each broker / candidate has private preferences and may game the allocator. The algorithmic question is *equilibrium selection*: which Nash equilibrium does the multi-agent learning dynamics converge to, and is it a desirable one?

1.1 Anchoring

A useful connection is iterative data collection: the policy changes the opponent distribution it trains against. AlphaZero is not a GAN: its value head predicts game outcomes and its search procedure does not play the role of a discriminator. Track the policy, search and value targets separately.

You already know MAPPO if you know PPO: it is PPO with a centralised critic that consumes the global state and per-agent actors that consume only local observations. Centralised critic, decentralised execution — the critic at training time sees everything, but at deployment each agent acts on local information alone.

1.2 Why this week sits where it does

Bandits (Week 11) and ranking (Week 12) are degenerate one-step games. This week we put the games back. The same machinery (UCB, posterior sampling, importance-weighted policy gradients) carries over with the extension that each agent's update must anticipate the others' updates. Self-play and PSRO are best-response *cascades*: I update my best response to your current policy, you update yours, repeat. Some exact finite-game procedures have convergence results; generic neural self-play and approximate best responses do not inherit them.

Common pitfall

The non-stationarity problem is the central pitfall of MARL. A naive extension of single-agent DQN to two agents (each running independent DQN) loses every theoretical convergence guarantee. Empirically it sometimes works, often does not. CTDE was invented to recover

stability: the critic can condition on joint information, but target policies continue to change and training is not automatically stationary.

2 Mathematical formulation

2.1 Stochastic games

Definition 2.1.1 (Stochastic game / Markov game)**.** A tuple $(N, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{r_i\}_{i=1}^N, \gamma)$. N agents act simultaneously in shared state s_t; the joint action $\mathbf{a}_t = (a_{1,t}, \ldots, a_{N,t})$ determines the next state $s_{t+1} \sim P(\cdot|s_t, \mathbf{a}_t)$ and reward $r_{i,t} \sim r_i(s_t, \mathbf{a}_t)$ for each agent.

Special cases:

- *Cooperative*: $r_i \equiv r$ for all i.
- *Zero-sum 2-player*: $r_1 + r_2 \equiv 0$.
- *General-sum*: no constraints.
- *Dec-POMDP*: cooperative + partial observability (each agent only sees o_i, not s).

Definition 2.1.2 (Nash equilibrium)**.** A joint policy $\boldsymbol{\pi}^\star = (\pi_1^\star, \ldots, \pi_N^\star)$ is a Nash equilibrium if for every agent i and every alternative policy π_i',

$$V_i^{\boldsymbol{\pi}^\star} \;\geq\; V_i^{(\pi_i', \boldsymbol{\pi}_{-i}^\star)}.$$

No agent can unilaterally improve by deviating.

Intuition. Nash is the right solution concept when agents are strategic. in cooperative settings, a joint optimum is a Nash equilibrium, but some suboptimal coordination points are also equilibria; not every failure is; in zero-sum it is tight (the minimax value). General-sum may have multiple Nashes and learning may oscillate among them.

2.2 Cooperative MARL: centralised critic, decentralised execution

The CTDE recipe:

- Each agent i has a *decentralised actor* $\pi_i(a_i \mid o_i, \theta_i)$ with parameters θ_i that depends only on its local observation.
- A *centralised critic* $V(s; \phi)$ or $Q(s, \mathbf{a}; \phi)$ consumes the global state (and possibly all agents' actions) and is used *only at training time*.
- At deployment, each agent runs only its decentralised actor; the critic is discarded.

The advantage of CTDE: the critic sees through the non-stationarity (it has all agents' actions as input, so the joint dynamics are Markov in its input space) while the deployed system remains decentralised (no shared network at inference time).

2.2.1 MADDPG (Lowe et al. NeurIPS 2017)

The first CTDE-style continuous-action algorithm. Per-agent deterministic policy $a_i = \mu_i(o_i; \theta_i)$, per-agent centralised critic $Q_i(s, \mathbf{a}; \phi_i)$ trained on the multi-agent Bellman:

Theorem 2.2.1 (MADDPG policy gradient, Lowe et al. 2017)**.** *The deterministic policy gradient for agent i is*

$$\nabla_{\theta_i} J(\mu_i) = \mathbb{E}_{s \sim D}\Big[\nabla_{\theta_i} \mu_i(o_i) \nabla_{a_i} Q_i(s, \mathbf{a})\big|_{a_i = \mu_i(o_i)}\Big].$$

Sketch. The single-agent DDPG result generalises directly: the deterministic policy gradient is the inner product of the policy's Jacobian with the critic's gradient. The only change in MARL is that the critic depends on $\mathbf{a}_{-i}$ (other agents' actions), which are not differentiated through. □ □

2.2.2 MAPPO (Yu et al. NeurIPS 2022)

A simple but powerful recipe: vanilla PPO, but with a centralised state-value critic. Per-agent actor takes o_i, the critic takes the global state s. The PPO objective is unchanged per agent:

$$L_i^{\text{CLIP}}(\theta_i) = \mathbb{E}\Big[\min\left(r_t(\theta_i)A_t^{\text{GAE}}, \text{clip}(r_t(\theta_i), 1-\epsilon, 1+\epsilon)A_t^{\text{GAE}}\right)\Big].$$

The advantage A_t^{GAE} is computed from the centralised value function. The critic is shared across agents (one network, fed each agent's transitions); the actor is per-agent (or can be shared too, with agent-id embedding).

Published evidence. Yu et al. evaluate PPO-based methods on particle-world environments, SMAC, Google Research Football and Hanabi. The comparisons are empirical and protocol-dependent; neither a theorem of dominance nor a MA-MuJoCo result follows from that evaluation. Source: https://arxiv.org/abs/2103.01955.

2.2.3 QMIX (Rashid et al. ICML 2018)

Value-decomposition: factorise $Q_{\text{joint}}(s, \mathbf{a}) = f(Q_1(o_1, a_1), \ldots, Q_N(o_N, a_N))$ where f is a monotone aggregator (a mixing network with non-negative weights). This makes *decentralised greedy decisions consistent with joint argmax*:

$$\arg\max_{\mathbf{a}} Q_{\text{joint}}(s, \mathbf{a}) = (\arg\max_{a_1} Q_1, \ldots, \arg\max_{a_N} Q_N).$$

Remark (QMIX monotonicity caveat). The monotone aggregator is a strict restriction: it cannot represent joint Q-functions where one agent's best action depends on another agent's action in a way that flips ordering (the canonical counter-example is a "XOR-coordination" game). For tasks like SimpleSpread / SMAC where optimal play has compatible per-agent argmax actions, QMIX works; for adversarial-coordination tasks, it fails.

2.3 Competitive MARL: self-play, fictitious play, PSRO

2.3.1 Self-play

Both agents share the same policy network; each is trained against opponents that are previous versions of the same network. Goes back to Tesauro's TD-Gammon (1992) and is the recipe behind AlphaGo, AlphaZero, AlphaStar, and OpenAI Five.

Convergence boundary. Generic last-iterate neural self-play has no unconditional Nash-convergence guarantee. Classical fictitious-play results concern specified finite games and empirical mixed strategies; realisability of a neural policy class alone does not supply exact best responses or the required learning dynamics.

2.3.2 Fictitious play

At iteration t, each agent plays the best response to the empirical distribution of the opponent's historical actions $\bar{\pi}_{-i}^{(t)} = \frac{1}{t}\sum_{k=1}^{t} \pi_{-i}^{(k)}$. For finite matrix games, the classical results concern convergence of empirical mixed play toward equilibrium in two-player zero-sum and identical-interest settings, not convergence of each selected action (Robinson 1951; Monderer & Shapley 1996).

2.3.3 Policy-Space Response Oracles (PSRO)

Definition 2.3.1 (PSRO, Lanctot et al. NeurIPS 2017)**.** Iterate: (i) compute a Nash equilibrium $\sigma^{(t)}$ on the current population of policies $\Pi^{(t)}$; (ii) for each agent, compute the best response to $\sigma_{-i}^{(t)}$ via single-agent RL; (iii) add the best responses to the population; repeat.

PSRO unifies self-play, fictitious play, and double-oracle algorithms. The inner "best response" step is the only RL step; the meta-game over the policy population is solved by classical game-theoretic methods (LP for zero-sum, replicator dynamics for general-sum).

Remark (Exact double-oracle versus practical PSRO). In a finite two-player zero-sum normal-form game, exact best responses and an exact restricted-game equilibrium let double-oracle terminate when no profitable deviation remains. Finite policy populations and approximate RL oracles can miss deviations. No generic $\widetilde{O}(1/\sqrt{T})$ neural PSRO convergence rate follows merely from calling PPO as the oracle. Source: https://arxiv.org/abs/1711.00832.

3 AlphaZero

This chapter is the heart of the lecture. AlphaZero (Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, Science 2018; *Mastering the game of Go without human knowledge*, Nature 2017) unifies MCTS, deep RL, and self-play into a single training loop that has produced super-human play in Go, Chess, Shogi, and (with extensions like MuZero) Atari.

3.1 The core idea

AlphaZero has three components:

(1) A policy/value network. A single network with two heads: $\pi(\cdot \mid s;\theta) \in \Delta(\mathcal{A})$ predicts a policy distribution; $v(s;\theta) \in [-1,1]$ predicts the expected outcome from s.

(2) Monte Carlo Tree Search. Given a root state, MCTS uses the network as a heuristic to do focused tree search. Output: a refined policy $\pi_{\text{MCTS}}(\cdot \mid s)$ that uses both learned evaluations and lookahead. A finite search with an imperfect network need not improve every state or produce a calibrated winning probability. Evaluate its decisions separately.

(3) Self-play training. Generate games by self-play using MCTS at each move. For each (state, π_{MCTS}, game-result) triple, train the network to predict π_{MCTS} from the policy head and the game-result from the value head:

$$\mathcal{L}(\theta) = -\sum_a \pi_{\text{MCTS}}(a) \log \pi(a \mid s;\theta) + (v(s;\theta) - z)^2 + \text{wd}(\theta).$$

The training loop is a self-improvement cascade: better network produces better MCTS produces better self-play data produces a better network.

3.2 MCTS with PUCT

Definition 3.2.1 (PUCT selection rule, Silver et al. 2017)**.** At each node n with children c, select the child $c^\star$ maximising

$$\text{PUCT}(n,c) \;=\; -Q(c) \;+\; c_{\text{puct}}\, P(c) \cdot \frac{\sqrt{N(n)}}{1+N(c)},$$

where $Q(c)$ is the mean action-value from previous simulations (negated because the child's Q is from the opponent's perspective), $P(c)$ is the prior probability from the policy network, $N(n)$ is the parent's visit count, $N(c)$ is the child's visit count, and c_{puct} is an exploration constant (typically 1–4).

Intuition. PUCT is UCB1 with two changes: (1) the bonus uses the *prior policy* as a multiplier (so the network's preferences guide exploration), and (2) the bonus shrinks as $\sqrt{N(n)}/(1+N(c))$ instead of $\sqrt{\log N/N(c)}$. The $\sqrt{N}$ in the numerator (rather than $\sqrt{\log N}$) gives *aggressive* exploration of the priors; empirically this is what makes AlphaZero work on board games.

Remark (PUCT is a search allocation rule)*.* The prior-weighted bonus allocates simulations; it is not an uncertainty interval. Do not attach a minimax bandit regret rate to this tree rule without specifying prior support, value estimation and the nonstationary returns induced by deeper search. This lecture asserts no such rate.

3.3 Dirichlet noise

At the root of every MCTS, AlphaZero adds Dirichlet noise to the prior:

$$P'(\cdot \mid s_{\text{root}}) = (1-\epsilon)P(\cdot \mid s_{\text{root}}) + \epsilon\,\eta, \quad \eta \sim \text{Dirichlet}(\alpha \mathbf{1}).$$

Typical values: the AlphaZero preprint uses $\alpha = 0.3$ for chess, 0.15 for shogi and 0.03 for Go, with root noise scaled to typical legal-action counts (https://arxiv.org/abs/1712.01815). The noise injects exploration at the root, preventing the network from collapsing onto a single move.

Common pitfall

Forgetting the Dirichlet noise is the most common AlphaZero bug. Without it, the policy network gets trained only on its own preferences, never explores alternatives, and converges to a sub-optimal fixed point. Root noise is one exploration mechanism; PUCT and sampling from root visits also affect which moves are explored.

3.4 Training loss

Theorem 3.4.1 (AlphaZero training loss)**.** *For each self-play tuple $(s, \pi_{\text{MCTS}}, z)$, the loss is*

$$\mathcal{L}(\theta; s, \pi_{\text{MCTS}}, z) = (z - v(s;\theta))^2 - \pi_{\text{MCTS}}^{\top} \log \pi(\cdot \mid s;\theta) + c\|\theta\|^2.$$

The first term regresses the value head to the game outcome; the second trains the policy head to match the MCTS visit distribution; the third is L2 regularisation.

Why these targets are useful. The policy head distils a finite search distribution; the value head learns the realised outcome from the stored player's perspective. Outcomes are samples under the data-generating self-play/search policies, which can change during training. This resembles generalised policy iteration, but neither finite search nor neural regression guarantees a monotone improvement or equilibrium strategy.

3.5 Symmetry augmentation

For Tic-Tac-Toe (and Go, and chess to a lesser extent), the game has geometric symmetries. AlphaZero augments each (state, π_{MCTS}, z) tuple with all D_4 symmetries (4 rotations $\times$ 2 reflections = 8 augmentations on a square board). This creates eight correlated transformations, not eight independent games, and adds training work. The local Tic-Tac-Toe demo uses D_4 symmetries; AlphaZero's cross-game implementation did not use the same augmentation recipe as AlphaGo Zero.

3.6 Putting it together

Algorithm 1: AlphaZero training loop

Input: Network $f_\theta = (\pi, v)$, MCTS budget n_{sim}, exploration constant c_{puct}, Dirichlet (ϵ, α).
Init: Replay buffer $\mathcal{D} = \emptyset$.
for *iteration* $= 1, 2, \ldots$ **do**
  for *self-play episode* **do**
    $s \leftarrow$ initial state
    while s *is non-terminal* **do**
      Run MCTS with f_θ from s for n_{sim} simulations (with Dirichlet noise at root).
      $\pi_{\text{MCTS}} \leftarrow$ normalised visit counts at root.
      Sample $a \sim \pi_{\text{MCTS}}^{1/\tau}$ (with temperature τ).
      For each symmetry g, store $(g \cdot s, g \cdot \pi_{\text{MCTS}})$ in $\mathcal{D}$.
      $s \leftarrow s'$ after applying a.
    For each stored tuple in this episode, set $z =$ game result from the player's perspective.
  for *training epoch* **do**
    Sample minibatch from $\mathcal{D}$.
    Take SGD step on $\mathcal{L}(\theta; s, \pi_{\text{MCTS}}, z)$.
  Evaluate f_θ vs. previous best; if winrate $>$ threshold, replace.

4 MuZero, MCTS extensions, and modern self-play

4.1 MuZero (Schrittwieser et al. Nature 2020)

The natural extension of AlphaZero to environments where the dynamics are unknown. MuZero learns a latent dynamics model alongside the policy and value: at each MCTS expansion, instead of applying the true environment transition, the network predicts a latent next-state and a reward. The tree search happens entirely in the latent space.

Theorem 4.1.1 (MuZero loss)**.** *Joint loss across K-step unrolls:*

$$\mathcal{L} = \sum_{k=0}^{K} \left[\ell^p(\pi_k^{\mathrm{MCTS}}, \pi_k) + \ell^v(z_k, v_k) + \ell^r(u_k, r_k)\right] + c\|\theta\|^2,$$

where ℓ^r is the model's reward-prediction loss and u_k is the observed reward at unroll step k.

Why MuZero matters. It unifies AlphaZero (board games) and Atari-style RL (no model) into a single framework. The latent dynamics model is trained *end-to-end with the policy and value*, so it learns exactly the features that matter for planning. MuZero achieves super-human play in Go / Chess / Shogi *and* sets new SOTA on Atari with the same algorithm.

4.2 Stochastic MuZero and EfficientZero

Stochastic MuZero (Antonoglou et al. 2022) extends to stochastic environments by sampling latent next-states. **EfficientZero** (Ye et al. NeurIPS 2021) adds a self-supervised consistency loss between predicted and actual next states, plus a search-policy entropy bonus; trains to near-MuZero Atari scores with $\sim 1/100$ the data.

4.3 KataGo and the AlphaZero hyperparameter zoo

KataGo (Wu 2019, ongoing) is the open-source Go engine descended from AlphaGo Zero. It has spent five years optimising the AlphaZero recipe: playout cap randomisation (small-search games for fast iteration, big-search games for label quality), targeted exploration of unbalanced positions, score-based auxiliary heads, transfer learning across board sizes. The net effect: KataGo at $\sim 10^4$ TPU-hours matches AlphaGo Zero at $\sim 10^6$ TPU-hours. The lessons:

- Mixing playout budgets across self-play games is a free win (Wu 2019); some games with 50 sims, some with 1000.
- Auxiliary heads (score, ownership, ranking) regularise the backbone and stabilise the policy / value heads.
- Symmetry augmentation is essential at small sizes (we exploit the same on Tic-Tac-Toe).

4.4 Self-play in two-player zero-sum games: theory

Proposition 4.4.1 (No regret in a finite zero-sum matrix game)**.** *If both players have external regrets $R_1(T), R_2(T)$, their average mixed strategies have saddle-point gap at most $(R_1(T) + R_2(T))/T$. Sublinear regrets therefore yield a vanishing gap. This concerns the* product of the average strategies*, not each last iterate or an arbitrary statewise average in a stochastic game.*

The result generalises to extensive-form games via Counterfactual Regret Minimisation (CFR; Zinkevich et al. NeurIPS 2007), which is the algorithm behind the super-human poker bots Libratus (Brown & Sandholm Science 2017) and Pluribus (Brown & Sandholm Science 2019).

4.5 League training (AlphaStar)

AlphaStar (Vinyals et al. Nature 2019) extends self-play to handle *non-transitive* games (StarCraft II, where rock-paper-scissors dynamics exist between unit compositions). The recipe:

- Maintain a *league* of past policies.
- Train "main agents" against the league with a prioritised sampling that focuses on the league's strongest exploiters.
- Periodically add the main agent's snapshot back to the league.
- Train "main exploiters" (focused on beating main agents) and "league exploiters" (focused on beating the league average).

League diversity broadens training opponents; measured robustness against an evaluation pool does not exhaust the entire strategy space.

5 Algorithms

Algorithm 2: MAPPO with centralised critic

Input: N agents, per-agent actor $\pi_i(a_i|o_i;\theta_i)$, centralised critic $V(s;\phi)$, PPO clip ϵ.
for *iteration* $= 1, 2, \ldots$ **do**
  `// Rollout`
  for *episode* **do**
    for $t = 1, \ldots, T$ **do**
      Each agent i samples $a_{i,t} \sim \pi_i(\cdot \mid o_{i,t})$.
      Step env: $s_{t+1}, \{r_{i,t}\} \leftarrow \text{env.step}(\mathbf{a}_t)$.
      Store $(o_{i,t}, a_{i,t}, \log \pi_i(a_{i,t}|o_{i,t}), r_{i,t}, V(s_t), s_t)$.
  `// GAE per agent`
  for *agent* $i = 1, \ldots, N$ **do**
    Compute $A_{i,t}^{\text{GAE}}$ from $(r_{i,t}, V(s_t))$.
  `// PPO update`
  for *epoch* $= 1, \ldots, K$ **do**
    Sample minibatch; compute L_i^{CLIP} and critic MSE.
    Minimise $-\sum_i L_i^{\text{CLIP}} + \frac{1}{2}(V(s) - R)^2$

Algorithm 3: Self-play in zero-sum games

Input: Initial policy $\pi^{(0)}$, training procedure $\text{Train}(\cdot, \cdot)$.
for *iteration* $t = 1, 2, \ldots$ **do**
  Sample games with $\pi^{(t-1)}$ playing both sides.
  $\pi^{(t)} \leftarrow \text{Train}(\pi^{(t-1)}, \text{games}_t)$.
  Evaluate $\pi^{(t)}$ vs. a fixed test opponent; track ELO climb.

Example 5.0.1 (A by-hand MCTS rollout on Tic-Tac-Toe)**.** Initial state (all empty), to-move = X. Suppose the network priors put $P(\text{centre}) = 0.5$, $P(\text{corner}) = 0.1$ each (4 corners), $P(\text{edge}) = 0.025$ each (4 edges). $c_{\text{puct}} = 1.5$.

Simulation 1. The literal formula with parent count zero gives zero bonuses, so it needs a tie rule. The local implementation uses $\max(1, \sum_c N(c))$ in the square root, making the prior break the initial tie. The centre bonus is $1.5(0.5) = 0.75$. After moving there, the child is O-to-move. Suppose its value is -0.1 from O's perspective. Store child $Q = -0.1$ and back up $+0.1$ to X.

Simulation 2. With one visit to that child, its parent-view score is $-(-0.1) + 1.5(0.5)\sqrt{1}/(1 + 1) = 0.475$; each unvisited corner scores $1.5(0.1) = 0.15$. Centre is selected again. The eventual visit distribution requires the subsequent leaf evaluations; it cannot be deduced from these two simulations. If counts later sum to N, normalise by that actual N.

Complexity. MCTS visits up to $O(n_{\text{sim}} \cdot \text{depth})$ tree edges. With cached node expansions, it usually makes at most one new leaf-network evaluation per simulation plus root expansion; traversal count is not network-inference count. Time the selected budget on the actual machine rather than assume a one-minute runtime. MAPPO scales linearly in number of agents (per-agent actor inference) and linearly in episode length.

Common pitfall

The number of MCTS simulations is the single most important hyperparameter in AlphaZero. Too few and the policy improvement is negligible (training slows to a crawl). Too many and you burn compute that could be spent on more self-play games. The standard rule of thumb: at least ~ 100 simulations per move; ~ 800 on the original AlphaGo Zero; ~ 30 for Tic-Tac-Toe is enough.

6 Implementation walkthrough

The mini-project lives in `project/` and ships:

- `tictactoe.py`: 3x3 Tic-Tac-Toe with canonical-form encoding, legal-move generator, terminal test, D_4 symmetries.
- `alphazero.py`: `AZNetwork` (small CNN), `MCTS` with PUCT and Dirichlet noise, self-play loop, training step, `evaluate_vs_random`.
- `simple_spread.py`: 3-agent cooperative-coverage env.
- `coord_game.py`: 2-agent matrix coordination game.
- `mappo.py`: per-agent actor, centralised critic, GAE, PPO clip.
- `train.py`: orchestrates AlphaZero + MAPPO runs.
- `solutions.py`: reference solutions for 10 workbook exercises.

6.1 The network

```python
class AZNetwork(nn.Module):
    def __init__(self, board_size=3, n_actions=9, hidden=32):
        super().__init__()
        self.conv1 = nn.Conv2d(2, hidden, 3, padding=1)
        self.conv2 = nn.Conv2d(hidden, hidden, 3, padding=1)
        self.policy_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(hidden * board_size * board_size, 64), nn.ReLU(),
            nn.Linear(64, n_actions))
        self.value_head  = nn.Sequential(
            nn.Flatten(),
            nn.Linear(hidden * board_size * board_size, 32), nn.ReLU(),
            nn.Linear(32, 1), nn.Tanh())
```

Two conv layers and two heads. Output: logits + tanh-clipped value in $[-1, 1]$.

6.2 MCTS core

```python
def _select_child(self, node):
    total_visits = sum(c.visits for c in node.children.values()) or 1
    best_a, best_score, best_c = -1, -1e18, None
    for a, c in node.children.items():
        u = self.c_puct * c.prior * math.sqrt(total_visits) / (1 + c.visits)
        score = -c.q + u
        if score > best_score:
            best_score = score
            best_a, best_c = a, c
    return best_a, best_c
```

Two implementation details worth flagging:

- $-Q$ rather than $+Q$: we are at the parent (player to move), the child's value is from the opponent's POV; we want the move that is *worst* for the opponent, i.e. *most negative* from their perspective, i.e. $-Q$ maximised.
- "$1 + N(c)$" in the denominator: when $N(c) = 0$, the bonus is $c_{\text{puct}} P \sqrt{N(\text{parent})}$, fine for new nodes.

6.3 MAPPO actor and critic

```
class Actor(nn.Module):
    def __init__(self, obs_dim, n_actions, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, hidden), nn.Tanh(),
            nn.Linear(hidden, hidden), nn.Tanh(),
            nn.Linear(hidden, n_actions))

class CentralisedCritic(nn.Module):
    def __init__(self, state_dim, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden), nn.Tanh(),
            nn.Linear(hidden, hidden), nn.Tanh(),
            nn.Linear(hidden, 1))
```

Two-layer MLPs with Tanh activations (more stable than ReLU for small-batch on-policy PPO).

6.4 The orchestration

```
out = train(seed=0, az_iters=12, mappo_episodes=60, coord_episodes=200)
# out["alphazero"]["winrates"] -> ~ [0.5, 0.65, 0.75, ..., 0.9]
# out["coord"]["returns"][-30:].mean() -> ~ 10.0
# out["mappo"]["returns"]  -> noisy
```

7 Empirical results

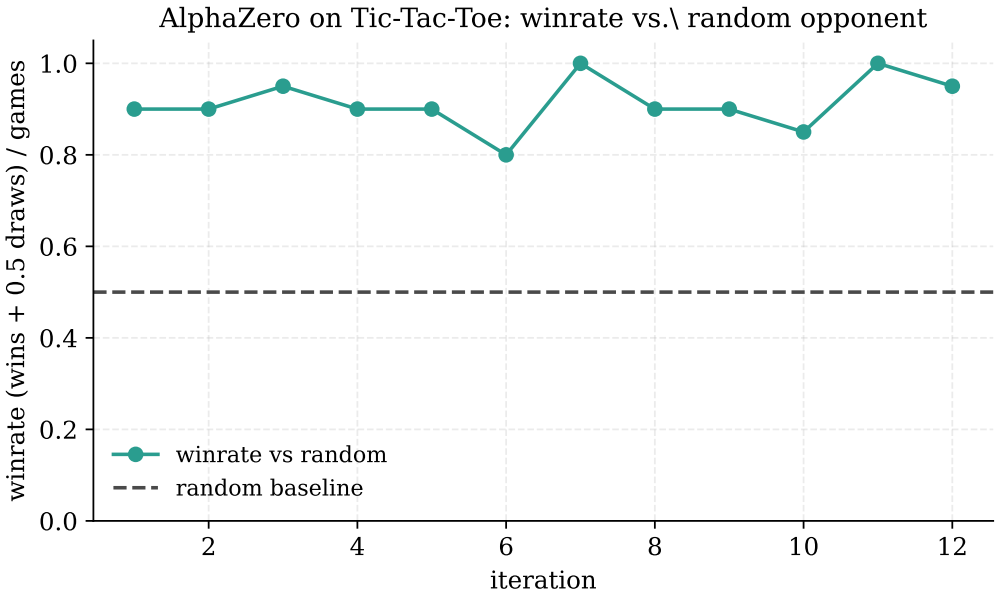


Figure 7.1: AlphaZero winrate vs. a uniform-random opponent on Tic-Tac-Toe. The network is a tiny CNN (32 hidden conv channels) trained for 12 iterations of 5 self-play episodes with 25 MCTS simulations per move. The legacy `winrate` field is score rate: wins plus half draws divided by games. Report wins/draws/losses and uncertainty separately; this curve is not an Elo rating or an optimal-play certificate. Note: against an optimal opponent, perfect play on TTT yields a draw.



Figure 7.2: AlphaZero training losses (policy CE + value MSE). Both decrease smoothly; the value loss plateaus once the network has memorised the terminal-value targets.

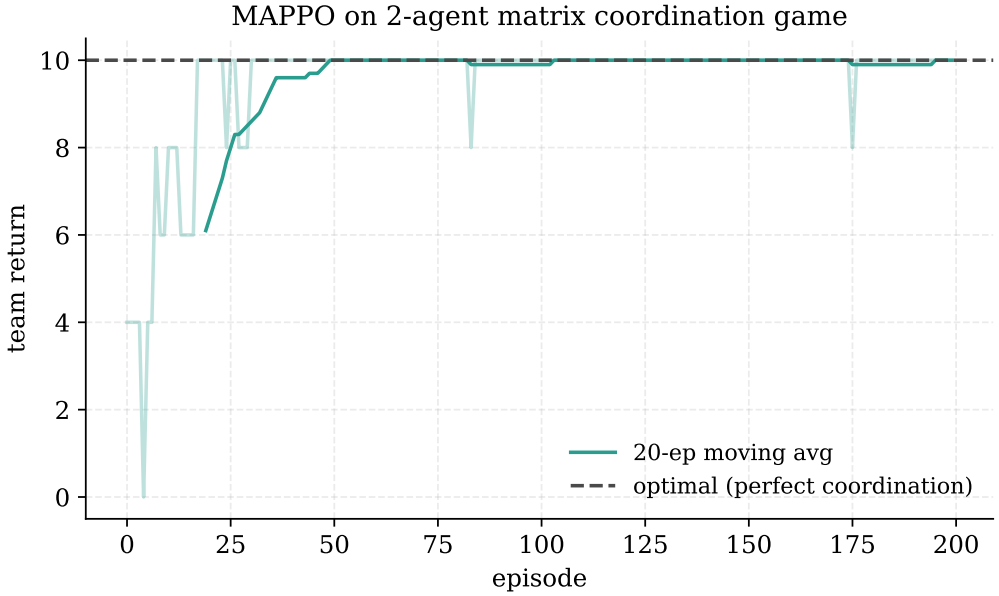


Figure 7.3: MAPPO on a 2-agent matrix coordination game (3 actions, 5-step episodes). Mean team return climbs to 10 (perfect coordination on every step) in ∼200 episodes.

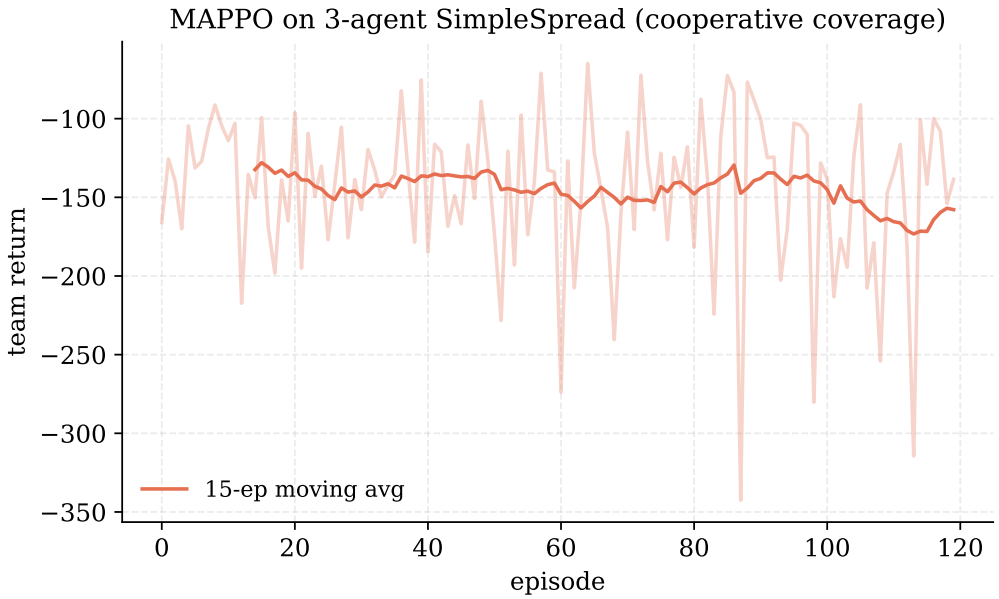


Figure 7.4: MAPPO on the 3-agent SimpleSpread cooperative-coverage problem. Return is noisy because of random initial agent / landmark placements; the moving-average curve shows mild improvement over 120 episodes. Published SimpleSpread benchmarks need $\sim 10^4$ episodes for full convergence; the figure is a snapshot of early learning.

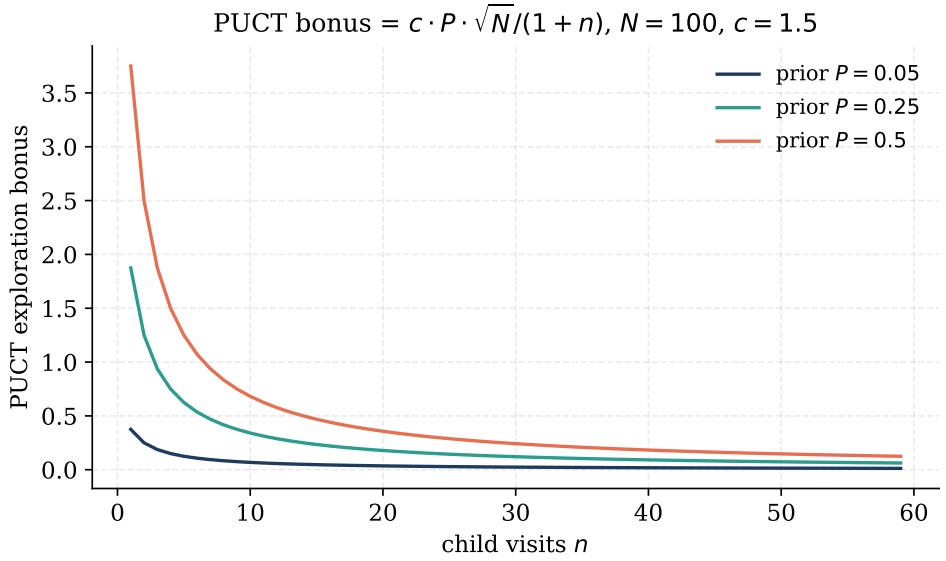


Figure 7.5: PUCT exploration bonus as a function of child visits, for three prior probabilities.

8 Safety angle

Safety angle

Self-play and MARL fail in characteristic ways:
F1 — Policy collapse. Self-play without Dirichlet noise (or similar exploration) converges to a degenerate fixed point where the network plays the same move regardless of state. *Mitigation:* Dirichlet noise at root (canonical AZ recipe); regular evaluation against a fixed test opponent; reject training updates that collapse the policy entropy below a threshold.
F2 — Cycle dynamics in general-sum MARL. In games without a unique Nash equilibrium (Rock-Paper-Scissors generalisations, cyclic adversarial dynamics), naive multi-agent learning cycles among the cycle of best responses without converging. *Mitigation:* fictitious play (average over historical opponent strategies, not just the latest); PSRO; explicit exploration regularisation.
F3 — Cooperative coordination failure. Multiple equivalent Nash equilibria with incompatible actions (canonical example: two cars at a 4-way stop — both go = crash, both stop = deadlock). Without explicit communication or a tie-breaking convention, agents may converge to a coordination-failure equilibrium. *Mitigation:* parameter sharing with agent/role information where needed; this can help learning but does not force compatible sampled actions; explicit communication channels; Schelling-style focal points.
F4 — Reward hacking via opponent modelling. In zero-sum settings the agent may learn to exploit a specific opponent's weaknesses rather than play robustly. The deployed system loses to opponents not in the training distribution. *Mitigation:* train against a diverse opponent population (PSRO is exactly this); evaluate against held-out opponents; use league training (AlphaStar) with explicit "main exploiters" and "league exploiters".
F5 — Information leakage into execution. Privileged information in the training-only critic is intentional CTDE and does not itself violate local execution. Test that actors receive only their permitted observations, that training and evaluation use the same actor inputs, and that no critic or future-label feature is accidentally passed to the deployed actor.
Two cites: Vinyals et al., *Grandmaster level in StarCraft II using multi-agent reinforcement learning*, Nature 2019 (AlphaStar league training); Schulman et al., *Proximal Policy Optimisation Algorithms*, arXiv:1707.06347 (PPO that underpins MAPPO).

9 Matching application

Matching application

Multi-agent and game-theoretic primitives apply to synthetic insurance / ATS in three distinct places:

Two-sided learning dynamics. synthetic insurance is a multi-agent system: the platform learns to allocate; the brokers learn to bid / archive. The allocation policy and the broker strategy co-evolve. A naive "platform optimises bind rate" policy plays into a general-sum game where brokers adapt their book of business and bidding to what the platform rewards. The Liu–Mania–Jordan AISTATS 2020 framework (Week 12) is the rigorous version of this dynamic; PSRO can explore policy responses, but stability in a matching market and Nash equilibrium of a specified game are distinct concepts.

Self-play for synthetic data. When real labels are scarce (early-stage synthetic insurance or a new ATS vertical), we can build a synthetic adversary that generates challenging leads, and train the allocator against it via self-play. AlphaZero-style policy iteration generates both the data and the learner; the value head is a fitted predictor that requires a separate calibration and distribution-shift audit.

Centralised training, decentralised execution. The synthetic insurance panel of brokers each runs their own scoring model (a recruiter desk runs its own shortlist policy). A CTDE-style architecture — a centralised value function during training that consumes the joint allocator + broker state, decentralised per-broker actors that consume only local information — is the right framework when brokers / recruiters have heterogeneous information.

Capstone connection (Week 24). The capstone safety-aware matching pipeline combines this week's CTDE machinery (centralised critic with full information for training stability) with Week 11's OPE (off-policy evaluation against the historical allocator), Week 12's fairness LP (post-processor), and Week 22's CMDP (constrained MDP for "never violate exposure floors"). The synthesis is the deliverable of Week 24.

10 Exercises

10 graded exercises (full statements in `workbook.ipynb`):

- **E1** ($\star$) PUCT formula.
- **E2** ($\star$) Tic-Tac-Toe terminal value.
- **E3** ($\star$) Enumerate the 8 D_4 symmetries.
- **E4** ($\star\star$) Run MCTS for 30 simulations from the empty board; inspect the visit distribution.
- **E5** ($\star\star$) Train AlphaZero for 5–8 iterations; report winrate vs random.
- **E6** ($\star\star$) Implement GAE; verify on a simple sequence.
- **E7** ($\star\star$) Train MAPPO on the coordination game; verify convergence to 10/10.
- **E8** ($\star\star$) Train MAPPO on SimpleSpread; observe the learning curve.
- **E9** ($\star\star\star$, open-ended) Compare self-play vs. "train against a fixed weak opponent"; argue why self-play produces stronger play.
- **E10** ($\star\star\star$, open-ended) AlphaZero "ELO climb": track winrate over iterations.

11 Further reading

The `extended_reading.pdf` document covers five papers: Silver et al. Nature 2017 (AlphaGo Zero), Silver et al. Science 2018 (AlphaZero), Yu et al. NeurIPS 2022 (MAPPO), Rashid et al. ICML 2018 (QMIX), Lanctot et al. NeurIPS 2017 (PSRO). Plus Vinyals et al. Nature 2019 (AlphaStar) and Schrittwieser et al. Nature 2020 (MuZero) as bonus.

Sourced from `research/deeprl_core_sota.md` sections covering MARL and self-play.